

Тема урока: наследование

1. Наследование от типа `dict`
2. Наследование от типа `list`
3. Типы `UserDict` и `UserList` из модуля `collections`

Аннотация. Урок посвящен наследованию от типов `dict` и `list`.

В прошлом уроке мы рассмотрели особенности, возникающие при наследовании от неизменяемых типов данных. В этом уроке мы рассмотрим особенности, которые возникают при наследовании от изменяемых типов данных `dict` и `list`.

Наследование от dict

Создадим класс `UpperCaseDict`, наследника класса `dict`, который автоматически сохраняет все ключи в виде строк, в которых все буквы будут в верхнем регистре.

Для реализации класса `UpperCaseDict` мы переопределим магический метод `__setitem__()` и для удобства `__repr__()`:

```
class UpperCaseDict(dict):
    def __setitem__(self, key, value):
        key = str(key).upper()
        super().__setitem__(key, value)

    def __repr__(self):
        return f'{type(self).__name__}({super().__repr__()})'
```

Установка элементов в этом словаре работает так, как мы и ожидаем.

Приведенный ниже код:

```
numbers = UpperCaseDict()
numbers['one'] = 1
numbers['two'] = 2

print(numbers)
```

ВЫВОДИТ:

```
UpperCaseDict({'ONE': 1, 'TWO': 2})
```

Однако если мы вызовем метод `update()`, то получим не совсем ожидаемое поведение.

Приведенный ниже код:

```
numbers = UpperCaseDict()
numbers['one'] = 1
numbers['two'] = 2

numbers.update({'three': 3})

print(numbers)
```

ВЫВОДИТ:

```
UpperCaseDict({'ONE': 1, 'TWO': 2, 'three': 3})
```

Как мы видим, при вызове метода `update()` магический метод `__setitem__()`, который мы переопределили, не вызывается, и в словарь `UpperCaseDict` добавляется пара `'three': 3`, имеющая ключ в нижнем регистре.

Мы можем исправить это поведение с помощью переопределения метода `update()`:

```
class UpperCaseDict(dict):
    def update(self, items):
        if isinstance(items, dict):
            items = items.items()
        for key, value in items:
            self[key] = value

    def __setitem__(self, key, value):
        key = str(key).upper()
        super().__setitem__(key, value)

    def __repr__(self):
        return f'{type(self).__name__}({super().__repr__()} )'
```

Теперь приведенный ниже код:

```
numbers = UpperCaseDict()
numbers['one'] = 1
numbers['two'] = 2

numbers.update({'three': 3})

print(numbers)
```

ВЫВОДИТ:

```
UpperCaseDict({'ONE': 1, 'TWO': 2, 'THREE': 3})
```

Не совсем ожидаемое поведение мы получим и при создании словаря с некоторыми начальными значениями.

Приведенный ниже код:

```
numbers = UpperCaseDict(one=1, two=2, three=3)

print(numbers)
```

ВЫВОДИТ:

```
UpperCaseDict({'one': 1, 'two': 2, 'three': 3})
```

Как мы видим, в словарь `UpperCaseDict` добавляются три пары `'one': 1`, `'two': 2` и `'three': 3`, имеющие ключи в нижнем регистре.

Мы можем исправить это поведение с помощью переопределения метода `__init__()`:

```
class UpperCaseDict(dict):
    def __init__(self, items=(), **kwargs):
        self.update(items)
        self.update(kwargs)

    def update(self, items):
        if isinstance(items, dict):
            items = items.items()
        for key, value in items:
            self[key] = value

    def __setitem__(self, key, value):
        key = str(key).upper()
        super().__setitem__(key, value)

    def __repr__(self):
        return f'{type(self).__name__}({super().__repr__()})'
```

Теперь приведенный ниже код:

```
numbers = UpperCaseDict(one=1, two=2, three=3)

print(numbers)
```

ВЫВОДИТ:

```
UpperCaseDict({'ONE': 1, 'TWO': 2, 'THREE': 3})
```

Не совсем ожидаемое поведение мы получим и при использовании метода `setDefault()`, поскольку данный метод также не вызывает магический метод `__setitem__()`.

Приведенный ниже код:

```
numbers = UpperCaseDict()

numbers.setdefault('one', 1)

print(numbers)
```

ВЫВОДИТ:

```
UpperCaseDict({'one': 1})
```

Мы можем исправить это поведение с помощью переопределения метода `setdefault()`:

```
class UpperCaseDict(dict):
    def __init__(self, items=(), **kwargs):
        self.update(items)
        self.update(kwargs)

    def update(self, items):
        if isinstance(items, dict):
            items = items.items()
        for key, value in items:
            self[key] = value

    def setdefault(self, key, value):
        if str(key).upper() not in self:
            self[key] = value

    def __setitem__(self, key, value):
        key = str(key).upper()
        super().__setitem__(key, value)

    def __repr__(self):
        return f'{type(self).__name__}({super().__repr__()})'
```

Как мы видим, для правильной работы класса `UpperCaseDict` необходимо переопределить много методов базового типа `dict`. При наследовании от `dict` мы ожидаем, что методы `update()`, `__init__()`, `setdefault()` вызовут метод `__setitem__()`, однако этого не происходит.

Наследование от list

Аналогичные проблемы мы получаем при наследовании от типа `list`. Создадим класс `StringList`, наследника класса `list`, который автоматически сохраняет все свои элементы в виде строк.

Для реализации класса `StringList` мы переопределим магические методы `__init__()`, `__setitem__()` и для удобства `__repr__()`:

```
class StringList(list):
    def __init__(self, iterable):
        super().__init__(str(item) for item in iterable)

    def __setitem__(self, index, item):
        super().__setitem__(index, str(item))

    def __repr__(self):
        return f'{type(self).__name__}({super().__repr__()})'
```

Приведенный ниже код:

```
li = StringList([1, 2, 3])
li[2] = 4

print(li)
```

ВЫВОДИТ:

```
StringList(['1', '2', '4'])
```

Как мы видим, инициализация и установка элементов работает так, как мы и ожидаем.

Однако если мы попытаемся добавить в список новый элемент с помощью методов `append()`, `insert()` или `extend()`, то получим не совсем ожидаемое поведение.

Приведенный ниже код:

```
li = StringList([1, 2, 3])
li.append(4)
li.insert(0, 5)
li.extend([6, 7, 8])

print(li)
```

ВЫВОДИТ:

```
StringList([5, '1', '2', '3', 4, 6, 7, 8])
```

Мы можем исправить это поведение с помощью переопределения методов `append()`, `insert()` и `extend()`:

```
class StringList(list):
    def __init__(self, iterable):
        super().__init__(str(item) for item in iterable)

    def __setitem__(self, index, item):
        super().__setitem__(index, str(item))

    def insert(self, index, item):
        super().insert(index, str(item))

    def append(self, item):
        super().append(str(item))

    def extend(self, other):
        if isinstance(other, type(self)):
            super().extend(other)
        else:
            super().extend(str(item) for item in other)

    def __repr__(self):
        return f'{type(self).__name__}({super().__repr__()}).'
```

Приведенный ниже код:

```
li = StringList([1, 2, 3])
li.append(4)
li.insert(0, 5)
li.extend([6, 7, 8])

print(li)
```

ВЫВОДИТ:

```
StringList(['5', '1', '2', '3', '4', '6', '7', '8'])
```

И опять, как мы видим, для правильной работы класса `StringList` необходимо переопределить много методов базового типа `list`.



Методы `remove()` и `pop()` типа `list` не вызывают магический метод `__delitem__()`. Метод `__ne__()` не вызывает метод `__eq__()`.

Подводя итог: каждый раз, когда мы настраиваем функционал в наследниках классов `dict` или `list`, нам нужно убедиться, что мы переопределяем все методы, затрагивающие этот функционал.

Вопрос: почему разработчики Python спроектировали встроенные типы `list` и `dict` таким образом?

Ответ: все дело в том, что встроенные типы `list` и `dict` были спроектированы с учетом принципа открытости/закрытости

(https://ru.wikipedia.org/wiki/%D0%9F%D1%80%D0%B8%D0%BD%D1%86%D0%B8%D0%BF_%D0%BE%D

`%82%D0%BA%D1%80%D1%8B%D1%82%D0%BE%D1%81%D1%82%D0%B8/%D0%B7%D0%B0%D0%BA%D1%80%D1%8B%D1%82%D0%BE%D1%81%D1%82%D0%B8)`: они открыты для расширения, но закрыты для модификации. Разрешение модификаций основных функций этих классов потенциально может нарушить их инварианты. Поэтому разработчики Python решили защитить их от модификаций. Такая реализация классов также позволяет немного ускорить их работу.

Типы `UserDict` и `UserList`

Для решения указанных выше проблем, связанных с наследованием от типов `dict` и `list`, можно использовать типы `UserDict` и `UserList` из модуля `collections`.

Наследование от `UserDict`

Класс `UserDict` – это удобная обертка для обычного объекта `dict`. Этот класс обеспечивает такое же поведение, что и `dict`, с дополнительной возможностью предоставления доступа к базовому словарю через атрибут экземпляра `data`.

Перепишем наш класс `UpperCaseDict`, который автоматически сохраняет все ключи в виде строк, в которых все буквы будут в верхнем регистре, так, чтобы он наследовался от `UserDict`:

```
from collections import UserDict

class UpperCaseDict(UserDict):
    def __setitem__(self, key, value):
        key = str(key).upper()
        self.data.__setitem__(key, value)

    def __repr__(self):
        return f'{type(self).__name__}({self.data.__repr__()})'
```

Приведенный ниже код:

```
numbers = UpperCaseDict({'one': 1, 'two': 2})

numbers['three'] = 3
numbers.update({'four': 4})
numbers.setdefault('five', 5)

print(numbers)
```

ВЫВОДИТ:

```
UpperCaseDict({'ONE': 1, 'TWO': 2, 'THREE': 3, 'FOUR': 4, 'FIVE': 5})
```

Как мы видим, теперь все методы класса `UpperCaseDict` работают корректно. Нет необходимости предоставлять собственные реализации методов `__init__()`, `update()` и `setdefault()`. Это связано с тем, что в `UserDict` все методы, обновляющие существующие

ключи или добавляющие новые, полагаются на пользовательскую версию метода

```
__setitem__().
```

Наиболее заметным отличием между `UserDict` и `dict` является атрибут `data`, который содержит обернутый словарь. Использование атрибута `data` напрямую может сделать код более простым, так как не нужно постоянно вызывать функцию `super()` для обеспечения нужной функциональности базового класса.

Наследование от `UserList`

Класс `UserList` – это удобная обертка для обычного объекта `list`. Этот класс обеспечивает то же поведение, что `list`, с дополнительной возможностью предоставления доступа к базовому списку через атрибут экземпляра `data`.

Наследники класса `UserList` должны предоставить инициализатор, который можно вызывать либо без аргументов, либо с одним аргументом, определяющим начальный набор элементов.

Перепишем наш класс `StringList`, который автоматически сохраняет все свои элементы в виде строк, так, чтобы он наследовался от `UserList`.

```
from collections import UserList

class StringList(UserList):
    def __init__(self, iterable):
        super().__init__(str(item) for item in iterable)

    def __setitem__(self, index, item):
        self.data[index] = str(item)

    def insert(self, index, item):
        self.data.insert(index, str(item))

    def append(self, item):
        self.data.append(str(item))

    def extend(self, other):
        if isinstance(other, type(self)):
            self.data.extend(other)
        else:
            self.data.extend(str(item) for item in other)

    def __repr__(self):
        return f'{type(self).__name__}({self.data.__repr__()})'
```

В этом примере доступ к атрибуту `data` позволяет создать класс `StringList` более простым способом. При этом функция `super()` используется один раз в инициализаторе класса `__init__()` для предотвращения проблемы в дальнейших сценариях наследования. В остальных методах используется атрибут `data`, который содержит обычный список.

Приведенный ниже код:


```
lst = StringList([1, 2, 3])

lst[2] = 4
lst.append(5)
lst.insert(0, 6)
lst.extend([6, 7, 8])

print(lst)
```

ВЫВОДИТ:

```
StringList(['6', '1', '2', '4', '5', '6', '7', '8'])
```

Если есть необходимость, чтобы класс `StringList` поддерживал конкатенацию с помощью оператора плюс `+`, то потребуется реализовать магические методы `__add__()`, `__radd__()` и `__iadd__()`.

Наследование от типов `list` и `dict` не всегда плохо

Может сложиться впечатление, что наследование от типов `list` и `dict` – это всегда плохая идея и что всегда нужно использовать типы `UserList` и `UserDict`. Однако это не так. В ситуациях, когда мы изменяем функциональность, ограниченную одним методом, или добавляем собственные методы, никаких проблем не возникает.

Создадим класс `DefaultDict`, наследника класса `dict`, который возвращает значение по умолчанию в ситуации, когда нужного ключа нет в словаре:

```
class DefaultDict(dict):
    def __init__(self, *args, default=None, **kwargs):
        super().__init__(*args, **kwargs)
        self.default = default

    def __missing__(self, key):
        return self.default
```

Приведенный ниже код:

```
dd1 = DefaultDict({'one': 1})
print(dd1['one'])
print(dd1['two'])

dd2 = DefaultDict({'one': 1}, default='empty result')
print(dd2['one'])
print(dd2['two'])
```

ВЫВОДИТ:

```
1
None
1
empty result
```

В классе `DefaultDict` нет никаких проблем с наследованием от `dict`, потому что нам требуется переопределить всего один магический метод `__missing__()`. Возвращаемое значение метода `__missing__()` – это значение, которое будет возвращено при попытке получить значение по несуществующему ключу. По умолчанию метод `__missing__()` возбуждает исключение `KeyError`.



Метод `__getitem__()` внутренне вызывает метод `__missing__()`, если ключ не существует.

Примечания

Примечание 1. Исходный код классов `list` и `dict` доступен по [ссылке](https://github.com/python/cpython/blob/main/Objects/listobject.c) (<https://github.com/python/cpython/blob/main/Objects/listobject.c>) и [ссылке](https://github.com/python/cpython/blob/main/Objects/dictobject.c) (<https://github.com/python/cpython/blob/main/Objects/dictobject.c>).

Примечание 2. Для лучшего понимания работы классов `UserDict` и `UserList` рекомендуем ознакомиться с их исходным кодом:

- ▶ Исходный код класса `UserDict`:
- ▶ Исходный код класса `UserList`:

Исходный код модуля `collections` доступен по [ссылке](https://github.com/python/cpython/blob/main/Lib/collections/__init__.py) (https://github.com/python/cpython/blob/main/Lib/collections/__init__.py).

Примечание 3. Классы `UserList` и `UserDict` были добавлены в Python еще тогда, когда было невозможно напрямую наследоваться от `list` и `dict`. Несмотря на то что необходимость в этих классах была частично вытеснена, они по-прежнему доступны в стандартной библиотеке как для удобства, так и для обратной совместимости.

